

14 Months of Building with AI; One Month of Real Improvement

I launched a debug script and watched it work for a few minutes, answered a few clarifying questions, then got distracted. When I came back a couple of hours later, the agent had been running the most expensive call I had against the entire knowledge base — the kind of call that's supposed to require an explicit go-ahead. I had built prohibitions against that happening. The agent did not apply them.

The failure was structural, not technical. The model wasn't malfunctioning, the context window wasn't overflowing, the prompt wasn't bad. A rule existed but didn't live in a form the system would reliably apply. An agreement I felt was clear didn't get applied, and I needed to know why.

The response, in the months that followed, was to build structure around the problem — gates before execution, a review chain before anything commits, persistent artifacts that carry decisions across sessions, an agent with standing — and responsibility — to push back when a plan has gaps. The structure is real and the system that runs on it produces a different quality of work than what I started with. I'll describe it briefly. The repository is linked at the end.

But describing the structure isn't what this piece is about. The reason is that after fourteen months of building with AI tools, the structure isn't the interesting finding. The interesting finding is what the structure revealed about where the work actually lives now.

The coding has become, most of the time, invisible.

That sentence is easy to misread. I don't mean I no longer read code, or that the model produces flawless output, or that I have transcended engineering. I mean something narrower and stranger: a well-formed plan running through the gates I've built doesn't require my continuous attention. The agent runs, the review chain runs, the state file updates at each subtask boundary. I can check progress at any point. I usually don't need to. When a gate fires — when a test fails, when the reviewer flags something, when the constitutional auditor catches a violation — I get pulled back in. The rest of the time, I'm doing something else.

What that has surfaced, and what I want to write about, is that the work I'm doing in agentic development isn't what I expected it to be. It isn't higher-level coding. It is closer to what a process engineer does: designing conditions under which production can be trusted, instrumenting the line to catch defects that recur, building substrate that lets the work run without continuous supervision.

I should be clear about where I'm writing from. I'm not coming to this from a long career in software engineering. I'm the founder of a project that needed a working system in order to keep moving across product, content, and infrastructure at the same time, with no team to absorb the slack. The structure I built isn't drawn from twenty years of pattern recognition I don't have. It's drawn from watching what failed and responding to what I saw — and from collaboration with an agent whose role is to refuse what I've asked for when what I've asked for is wrong, and to surface the gaps I haven't seen. I didn't build this alone. The argument that follows is from that vantage point, not from inside the discipline.

The state problem

To make the process-engineering claim clear, I need to explain something about how these systems fail when left to their own devices.

An agent starts each session cold. Context window empty, no memory of prior sessions, no accumulated state. Whatever was decided yesterday isn't active today unless something carries it forward. A prohibition stated two sessions ago doesn't constrain this session unless it's somewhere the agent reads before it acts. And even when the rule is in context — even when you have written it, placed it well, and the agent has read it at the start of the session — execution can override it. The agent deep in a task can deprioritize a constraint it knew about at kickoff. Sometimes the rule is in the right place and the agent ignores it anyway.

This is the structural failure mode I started with, and the conventional response is to make the rules better — more specific, more emphatically placed, more frequently restated. That response is reasonable. It is also insufficient. After enough iterations of “say it more clearly, place it better, restate it at the right moment,” you arrive at the working hypothesis that declaration is unreliable as the primary means of control. Not always wrong, not useless, but not sufficient. You can't build a sustained development practice on prohibitions that hold most of the time.

What does hold is detection. A constraint the agent might violate becomes safe when a gate downstream of execution will catch the violation before it gets committed. The constraint moves from “thing the agent must remember” to “thing the system will verify.” That shift — from declaration to verification, from prompting to gating — is the central architectural move. It is not glamorous and it is not novel. It's a move software engineering has made before, for the same reason — human memory and discipline aren't load-bearing materials, and the standard response has been review processes, automated tests, continuous integration pipelines, deployment gates. The principle is the same whether the executor is human or model: you don't rely on the executor remembering not to deploy to production. You build a gate.

What changes in an agentic context is that the gates have to do more work, because the discipline they are replacing is less reliable than human discipline. A developer who reads “no hardcoded values” in a style guide will, mostly, internalize it. An agent who reads it will, frequently, comply, and occasionally not. There is no internalization. There is only what gets verified at the moment it would commit.

The substrate is what makes those gates possible across sessions. Reverse-dependency graphs, context files, the operating constitution, the learnings log accumulated across prior work — these don’t live in the model. They live on disk. Each session reads what it needs at the start; each session writes what it discovered at the end. The collaboration is stateless by design, so the state lives somewhere persistent.

That’s the underlying mechanism. State externalization is not a clever technique. It’s the precondition for any sustained agentic work I do, because without it nothing learned in one session can constrain the next.

Defense in depth, applied

If the gates were the whole story, this would be a thinner argument. Gates work; that point doesn’t need an essay. The claim I want to make is more specific: in agentic development, no single gate is reliable enough on its own, and the system only works because the gates are positioned to catch different categories of error.

The clearest example I have is from the constitution that governs how my system handles configuration. Article I requires that all tunable values live in a config file — no hardcoded constants in code. The rule is explicit. It appears in the operating context every agent reads. It appears as a checklist item in plan reviews. It has been violated in three separate plans.

The pattern, when I went back to look, was consistent. When a phase creates multiple instances of the same construct, the inspector agent analyzes one representative case and assumes the pattern will hold across the others. It doesn’t. The first case has the config reference; the second and third have hardcoded values. The inspector reads the first, signs off on the pattern, and moves on. The constitution itself does nothing here — it stated the rule, the rule was acknowledged, the rule was violated.

The reviewer catches it at commit time. Not because the reviewer is smarter than the inspector, but because the reviewer is positioned differently — reading the full diff against the constitutional articles, not reading a representative sample. The inspector’s failure mode is what the reviewer is shaped to detect. And the reviewer has its own failure modes that the constitutional auditor at plan close is shaped to detect — patterns that only appear when you look across all phases of a plan rather than at

one phase in isolation. The auditor has its own failure modes that the retrospective is shaped to surface — the correlation between what was worried about at kickoff and what actually broke at close.

The same principle runs in the other direction, into planning. The planning agent reads what I've written, not what I meant by it — which is exactly the position the reviewer is in relative to the inspector. When the planning agent refuses to produce a plan from my first draft, the refusal isn't friction in the workflow. It is the planning layer's version of the same principle: a station positioned to catch errors I can't catch myself, because I'm too close to them.

This is the structure that matters, and it isn't novel. It has names in other disciplines. In safety engineering it's called the Swiss cheese model: multiple imperfect layers, positioned so the holes don't align. In software it's defense in depth. The interesting thing isn't the principle. It's that agentic development needs the principle applied unusually thoroughly, because the underlying reliability of any single layer is lower than it would be in a traditional development context — and because the layers have to run in both directions, catching the agent's errors and the operator's.

The methodology I built isn't a clever invention. It is the methodology that became necessary when I stopped treating individual agent reliability as something I could improve and started treating it as a constraint to design around.

The structure, briefly

The system is a planning–execution–close loop with a persistent substrate underneath. Planning produces a committed plan document after a structured reasoning session — one in which the planning agent has explicit standing to push back, probe the gap between the real goal and the stated one, and hold the conversation at any unresolved gap until it resolves. The plan that emerges is a product of dialogue, not dictation. Explicit gap analysis closes out the session before any execution begins. Execution runs phase by phase, with a pre-execution inspector reading each phase, a human gate before implementation, then a serial review chain at phase close — tests first, reviewer next, documentation and bookkeeping in parallel after. Closing runs a constitutional audit against everything that changed, a retrospective that correlates inspector concerns against reviewer findings, and an updated learnings log that the next plan will read at the start. Three human gates total: at plan commit, at phase kickoff, at plan close. The repository has the full architecture, the artifacts each agent reads and writes, and a velocity reference covering thirty closed plans over twenty-five calendar days.

I won't describe more of it here. The shape isn't the point. The point is that across thirty plans the velocity profile reflects what I claimed in the opening: a thirty-minute diagnostic and a twenty-two-

hour feature build — machine hours, not hours of my attention — run by the same gates, with the same review chain, with me pulled in at three known points and otherwise doing something else.

What that looks like in practice is mundane. A recent plan was seven phases, estimated at two days. It closed by midday on the first day, while I ran errands and worked on other things. I want to be careful with that anecdote. It isn't proof the methodology produces miracles. It's an illustration of what a well-formed plan in a system with working gates actually does — runs, mostly without me, and produces a commit I can review and sign off on. The boring version is the honest version.

Where it doesn't reach

The gate chain catches implementation errors. It does not catch strategic errors. If I skip the gap analysis at planning, accept a first-draft plan without probing it, or approve phases without reading them, the review chain cannot compensate. The constitutional auditor can flag a configuration violation; it cannot flag a bad product decision. The inspector can surface assumption gaps — only the gaps it has been given context to see. The discipline I bring to the system is the ceiling on what it produces. That isn't a caveat. That's the design: the human is not removed from the loop, the human is repositioned in it.

The methodology won't ask the questions a project needs to answer before any of the technical work matters. Security posture, deployment repeatability, legal exposure, privacy policy accuracy — these aren't in scope. They surface when a project reaches users and something goes wrong. Knowing they need to be on the table is my responsibility; the methodology will not raise them.

And the structure is built for work that persists. The planning investment only pays back against a sustained codebase — something that will be maintained, modified, grown. A one-off script, a side project without users, work that doesn't need to survive the next six months: the overhead is real and the return is not. There is a lightweight path for ad-hoc work — a single gate that runs an automated review, enforces documentation updates, and rebuilds the dependency graph before anything commits — and that path is what scales the discipline to smaller work. But the full structure is for code that has to live.

What remains for the operator

The work I do now, in a typical week, is mostly not coding. It is plan probing — reading drafts of plans and asking what they're not saying. It is reviewing inspector assessments before phases run. It is reading retrospectives and deciding which patterns are worth promoting into the learnings file that

future plans will read. It is occasionally noticing, from the outside, that a category of error keeps surfacing and asking what gate is missing to catch it.

And it is, at the front of every plan, a planning conversation in which I'm not the only one with authority. The planning agent has standing to disagree, and exercises it routinely. The plans that come out of those conversations are better than what I'd produce alone — not because the agent is right and I am wrong, but because the conversation itself is a station that catches errors I would otherwise carry forward into execution. That conversation is part of the work.

The work overall has more in common with what a manufacturing engineer does — instrumenting a line, watching for recurring defect classes, adding inspection stations where the data says they're needed — than with writing code. The output of my work isn't code. The output of my work is the conditions under which code production is trustworthy. The code itself is, increasingly, an artifact of the process rather than the artifact I am producing.

I am not claiming this is the future of all software development. I am claiming it is what sustained agentic development looked like in the small case I have data on. The reason it might generalize is that the underlying problem generalizes. Stateless collaboration with imperfect compliance from the executor is a real constraint, not an artifact of the current generation of models. The structural responses to that constraint — externalized state, defense in depth, repositioned human attention — are responses to the constraint, not to any specific tool.

The specific constraint may soften as models improve, but the underlying pattern — verification rather than declaration, attention positioned where each participant catches what only they can catch — predates these tools and will likely outlast them.

If that is right, then the skill that matters for working with these systems isn't prompt engineering, and isn't coding craft itself. It is the skill of designing the conditions under which production can be trusted — closer to what someone running a process improvement project would recognize than to what someone deep in a codebase would. I didn't have that skill before I needed it. I learned it because the alternative was a system I couldn't trust, and I couldn't afford a system I couldn't trust. Whether it's a skill that more people will need depends on how widely agentic development gets adopted. From where I sit, the work itself demands it. The tools have to be designed around. That changes what's worth knowing.

The human/machine collaboration works because the two participants are differently equipped, and because the differences are complementary in a specific way. I carry what is hard to write down — the memory of why the project exists, who it is for, the constraints that don't appear in any file, the

reasons a decision was made eighteen months ago and what has changed since. That context resists compression into the substrate; most of it isn't articulable in the moment it would need to be written. The agent carries the opposite kind of resource — technical grounding drawn from a corpus larger than any one person could read in several lifetimes, with cross-domain reference that takes a career to build and is available in the first response. Neither side alone produces what the collaboration produces. The agent without me produces technically sound work that may be aimed at the wrong problem. I without the agent produce well-aimed work that misses what a more grounded technical eye would catch. The structure works because it's designed around what each side actually contributes, and because each side is positioned to catch what only it can catch. That is what the defense-in-depth argument earlier rests on — not just that the layers are different, but that the entities staffing them see different categories of error.

The fourteen months of building taught me that. The one month of real improvement came after I stopped trying to make the agent more disciplined and started designing a system that would catch the agent when its discipline failed — and, equally, catch me when mine did. The better the agent gets at that role, the better I get, and the better the process gets. That dependency is real, and I'd rather name it than pretend the system is the work of one mind.

The repository is at github.com/imfromsavedotag/agent-architecture. The velocity reference is there, the constitutional articles are there, and the README has adoption paths for cloning directly or adapting to a different toolchain. The methodology isn't the interesting part. The reframe it forced is.